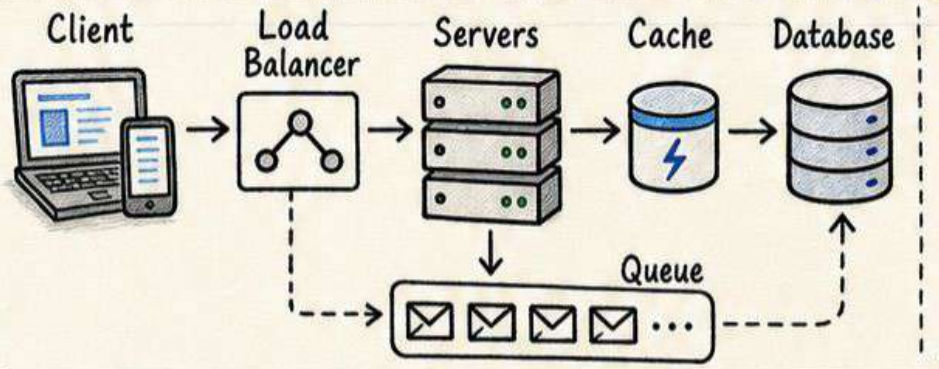
@ codedsoul_05

INTERVIEW PREP v1.0 ★

TOP 50 SYSTEM DESIGN INTERVIEW QUESTIONS

— WITH SHORT ANSWERS —

Everything asked in real backend interviews, on 9 pages. ☺





"Know the Concept.
Explain the Trade-off.
Draw the Diagram."

Concepts • Scaling • Databases • Caching • Queues • Reliability

Question Sections

Q1–7	Fundamentals
Q8–14	Scaling & Load Balancing
Q15–21	Databases & Storage
Q22–27	Caching
Q28–33	Messaging & Async
Q34–40	Distributed Systems
Q41–46	"Design a ..." Questions
Q47–50	Reliability & Ops

How to Answer

1. Clarify the scope
2. Estimate the scale
3. Define APIs & data
4. Draw the architecture
5. Deep dive one component
6. Handle failures
7. State the trade-offs

You'll Learn

- ☑ CAP & Consistency
- ☑ Sharding & Replication
- ☑ Load Balancing
- ☑ Caching Strategies
- ☑ Message Queues
- ☑ Rate Limiting
- ☑ Idempotency
- ☑ Consistent Hashing
- ☑ Microservices
- ☑ Observability
- ☑ Failure Handling
- ☑ Trade-off Thinking

Who is this for?

- Backend & SDE Interviews

"Interviewers do not want the perfect design.

SECTION 1 — FUNDAMENTALS

These decide whether the interviewer trusts the rest of your answer.

1



Q1. What is scalability?

The ability to handle more load by adding resources.
Vertical = bigger machine (limited).
Horizontal = more machines (preferred, needs statelessness).

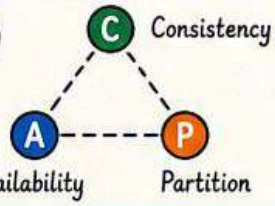
2



Q2. Latency vs Throughput?

Latency = time for one request.
Throughput = requests handled per second.
Batching raises throughput but usually raises latency too.

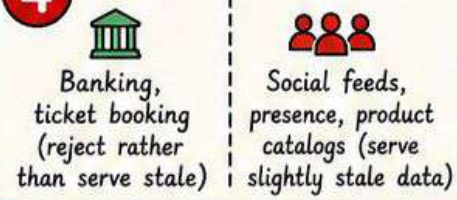
3



Q3. What is the CAP theorem?

During a network Partition you must choose
Consistency or Availability.
Not a permanent choice — it applies only while partitioned.

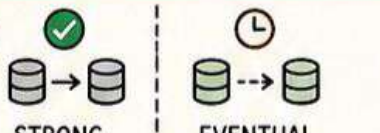
4



Q4. CP vs AP with examples?

CP — banking, ticket booking
(reject rather than serve stale).
AP — social feeds, presence, product catalogs
(serve slightly stale data).

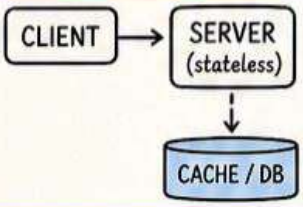
5



Q5. Strong vs eventual consistency?

Strong = every read sees the latest write (costly, slower).
Eventual = replicas converge over time (fast, cheaper).
Pick per feature, not per system.

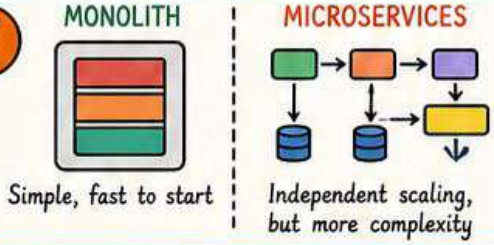
6



Q6. Stateless vs stateful services?

Stateless servers hold no session, so any instance
can serve any request — easy to scale and replace.
Push state to a cache or database.

7



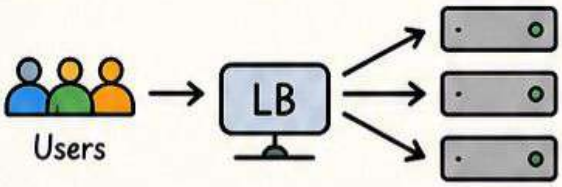
Q7. Monolith vs microservices?

Monolith is simpler and faster to start.
Microservices scale teams and components
independently, but add network calls,
partial failures and operational cost.

SECTION 2 — SCALING & LOAD BALANCING

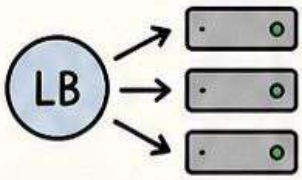
Scaling questions are really questions about bottlenecks.

Q8. What does a load balancer do?



A: Spreads traffic across healthy servers, removes failed ones via health checks, and hides individual instances behind one address.

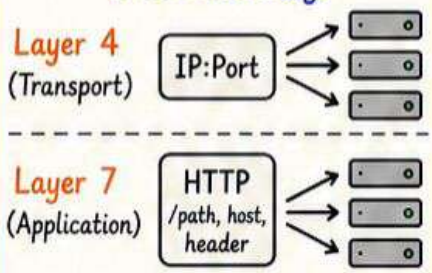
Q9. Common load balancing algorithms?



- Round robin
- Weighted round robin
- Least connections
- Least response time
- IP/consistent hash for stickiness.

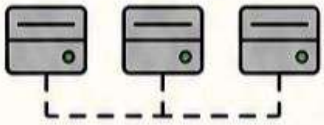
A: Round robin, weighted round robin, least connections, least response time, IP/consistent hash for stickiness.

Q10. Layer 4 vs Layer 7 load balancing?



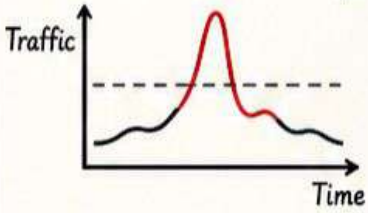
A: L4 routes on IP/port — fast and protocol-agnostic. L7 reads HTTP — can route by path, host or header, and terminate TLS.

Q11. What is horizontal scaling and what breaks it?



A: Adding more servers. It breaks when servers hold local state, when the database is a single writer, or when a shared lock serialises work.

Q12. How do you handle a sudden traffic spike?



A: Autoscale on a leading metric

- Queue non-urgent work
- Cache aggressively
- Shed low-priority traffic
- Pre-warm capacity for known events

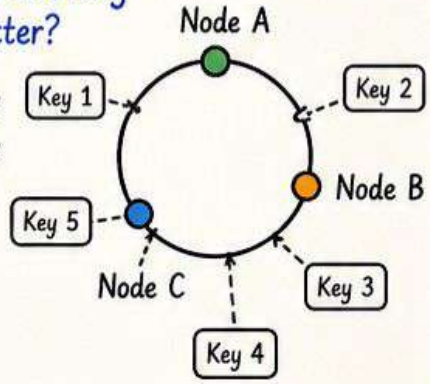
Q13. What is a CDN and when do you use it?



A: Edge servers that cache static and media content near users. Use it for images, video, JS/CSS — it cuts latency and offloads origin bandwidth.

Q14. What is consistent hashing and why does it matter?

A: Maps keys and nodes onto a ring adding or removing a node moves only a small slice of keys, instead of remapping everything. Virtual nodes even out the distribution.



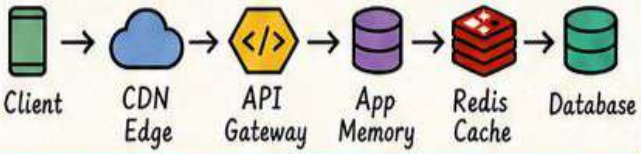
INTERVIEW TIP: Always name the bottleneck before scaling it —

SECTION 4 — CACHING

Caching is the cheapest performance win and the easiest correctness bug.

22 Q22. Where can you cache?

A: Client, CDN edge, API gateway, application memory, distributed cache (Redis), and inside the database itself. Each layer removes load from the next.

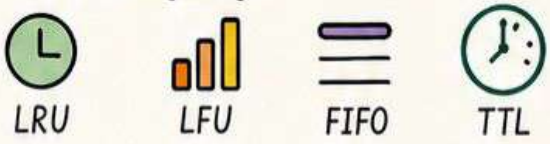


23 Q23. Cache-aside vs write-through vs write-back?

A: Cache-aside — app loads on miss and fills the cache (most common).
Write-through — write to cache and DB together (consistent, slower).
Write-back — write to cache, flush later (fast, risk of loss).

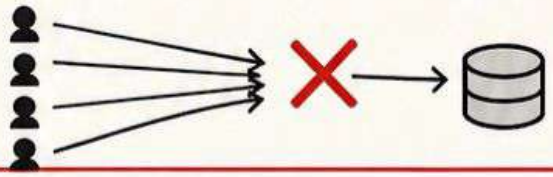
24 Q24. What are the main eviction policies?

A: LRU (least recently used) is the default choice, LFU for stable popularity, FIFO for simplicity, plus TTL expiry on every key.



25 Q25. What is cache stampede and how do you prevent it?

A: A popular key expires and thousands of requests hit the database at once. Fix with a lock or single-flight refresh, staggered TTLs (jitter), and background refresh before expiry.



26 Q26. Cache penetration and cache avalanche?

A: Penetration — requests for keys that do not exist; fix by caching a negative result or using a bloom filter.
Avalanche — many keys expire together; fix with randomised TTLs.



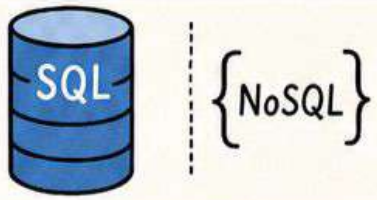
27 How dow do you invalidate a cache?

A: TTL expiry, explicit delete on write, or versioned keys (user:42:v3). Versioning avoids race conditions because you never mutate an existing entry.



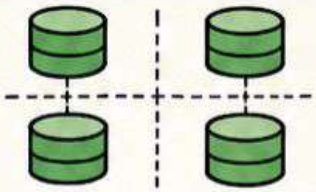
SECTION 3 — DATABASES & STORAGE

Choose storage from access patterns, never from popularity.



Q15. SQL vs NoSQL — how do you choose?

A: SQL for relations, joins and transactions. NoSQL for huge write volume, flexible schema and simple key-based access. Many systems use both.



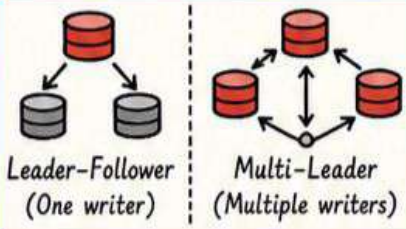
Q16. What is sharding?

A: Splitting data across databases by a shard key so each node holds a subset. Increases write and storage capacity but makes cross-shard joins hard.



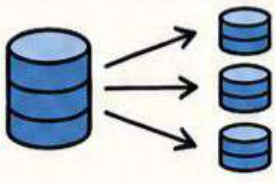
Q17. How do you pick a shard key?

A: High cardinality, evenly distributed, and matching the main query. Bad keys create hot shards — for example sharding by country or by timestamp.



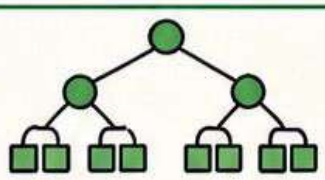
Q18. Replication — leader-follower vs multi-leader?

A: Leader-follower: one writer, many readers, simple and consistent. Multi-leader: writes in several regions, lower latency but conflicts must be resolved.



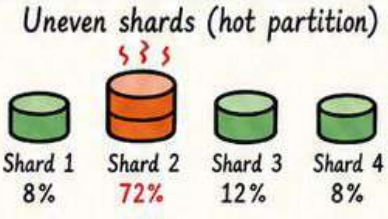
Q19. How do you scale reads?

A: Read replicas, caching, denormalisation and materialised views. Remember replicas lag — read-your-own-writes may need the primary.



Q20. What is an index and what does it cost?

A: A sorted structure (usually B-tree) that turns a scan into a lookup. Speeds up reads, slows down writes, and consumes storage.



Q21. What is a hot partition and how do you fix it?

A: One key or range gets a disproportionate share of traffic — a viral post or a stadium location. Fix by adding a time or random suffix to the key, splitting the range, caching it, or isolating it.

SECTION 5 — MESSAGING & ASYNC

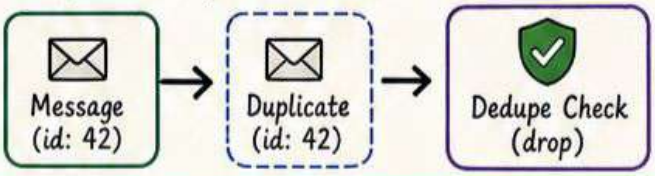
Queues turn a traffic spike into a longer queue instead of an outage.

Q28. Why use a message queue?

A: To decouple producers from consumers, absorb bursts, retry safely, and move slow work (emails, thumbnails, analytics) off the request path.

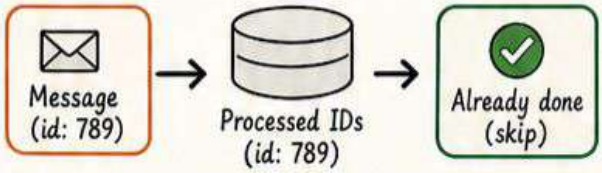
Q29. Delivery guarantees — at-most-once, at-least-once, exactly-once?

A: At-most-once may lose messages. At-least-once may duplicate. True exactly-once end-to-end is impractical — use at-least-once plus idempotent consumers.



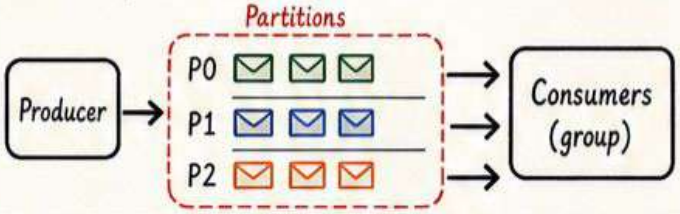
Q30. How do you achieve idempotency?

A: Give each message a unique id or business key, record processed ids, and make handlers safe to re-run — for example upserts instead of blind inserts.



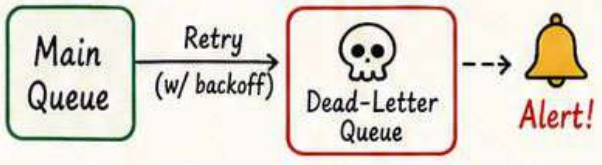
Q31. How do you preserve ordering?

A: Partition by a key (user_id, conversation_id) so related messages land on the same partition. Global ordering across all partitions is not scalable.



Q32. What is a dead-letter queue?

A: A separate queue for messages that failed after bounded retries. It keeps the main queue flowing and preserves the failure for investigation. Always alert on it.



Q33. Kafka vs RabbitMQ (or SQS)?

- Kafka — a durable partitioned log, replayable, huge throughput, consumers track offsets.
- RabbitMQ/SQS — classic broker semantics, per-message acks, easier routing, simpler for task queues.

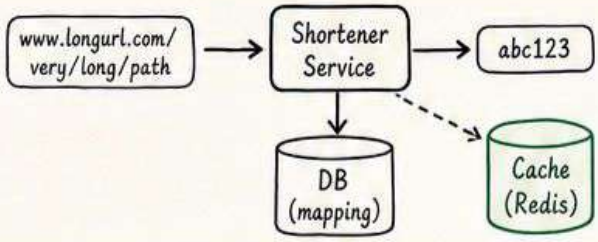


SECTION 7 — 'DESIGN A ...' QUESTIONS

For each one, remember the single hardest problem it hides.

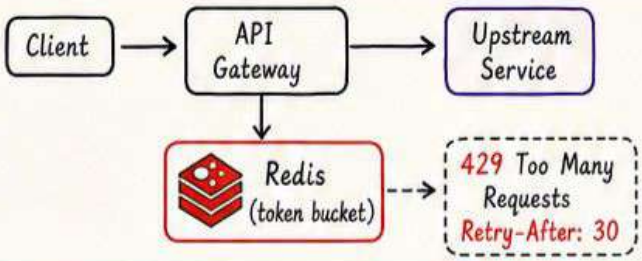
Q41. Design a URL shortener.

A: Hard part is unique short keys. Use base62 of a distributed counter or a hash with collision check. Read-heavy, so cache aggressively and serve redirects with a 301/302 from the edge.



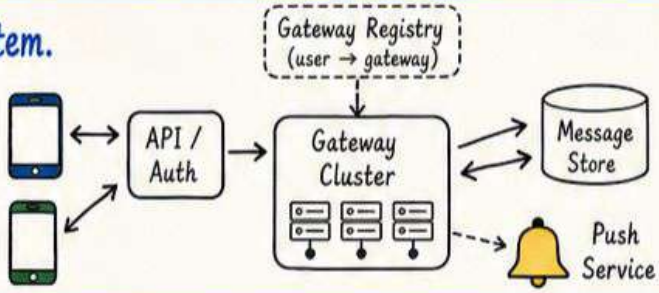
Q42. Design a rate limiter.

A: Token bucket in Redis keyed by user or API key, with atomic Lua updates. Return 429 with Retry-After. Discuss fixed window vs sliding window vs token bucket.



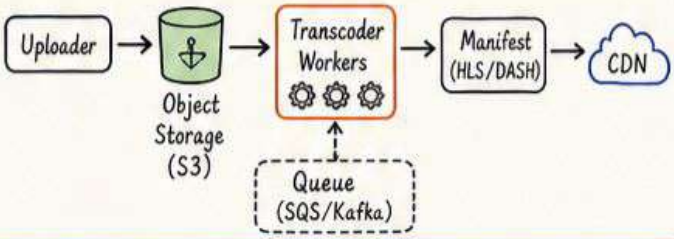
Q43. Design WhatsApp / a chat system.

A: Persistent WebSocket connections, a gateway registry mapping user to gateway, durable message store, at-least-once delivery with per-chat sequence numbers, and push notifications for offline users.



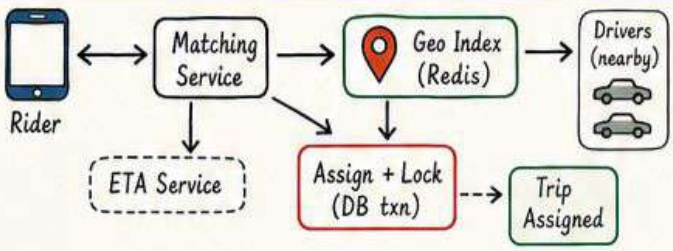
Q44. Design YouTube / a video platform.

A: Chunked resumable uploads to object storage, asynchronous transcoding into a bitrate ladder, HLS/DASH segments, and a global CDN. Read-heavy — optimise playback first.



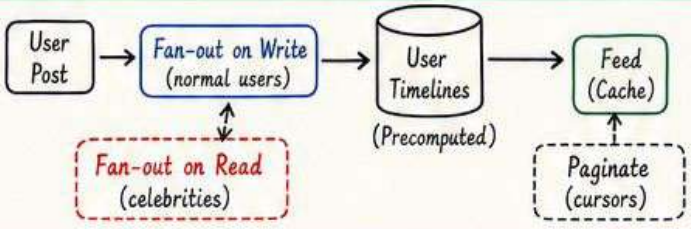
Q45. Design Uber / ride hailing.

A: Geo index (geohash or H3) in memory for driver positions, nearby search with neighbour cells, ranking by real ETA, and a lock plus transactional assignment so one driver is never double booked.



Q46. Design a news feed.

A: Fan-out on write for normal users (precomputed timelines), fan-out on read for celebrities with millions of followers. Hybrid is the correct answer. Cache the timeline, paginate with cursors.



SECTION 6 — DISTRIBUTED SYSTEMS

Every distributed question is really about failure and coordination.

34



Q34. What is a distributed lock and when do you need one?

A: A short-lived claim (usually in Redis with SET NX and a TTL) so only one worker acts on a resource. Always set an expiry, and confirm the result with a conditional database update.

35



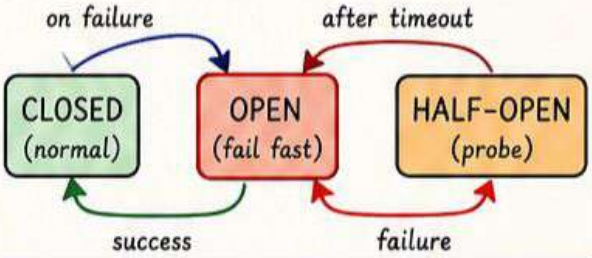
Q35. How do you prevent double booking or double charging?

A: An idempotency key plus a compare-and-set update: UPDATE ... WHERE status = 'available'. If zero rows change, someone else won the race.

36

Q36. What is a circuit breaker?

A: A wrapper that stops calling a failing dependency. States: CLOSED → OPEN (fail fast) → HALF-OPEN (probe a few requests). It prevents cascading failure.



37



Q37. Retry strategy — what can go wrong?

A: Naive retries amplify an outage. Use bounded attempts, exponential backoff with jitter, retry only transient errors, and never retry a non-idempotent call without a key.

38



Q38. What is a heartbeat and a TTL used for?

A: Detecting liveness. A node or connection registers with a TTL and refreshes it; if the heartbeat stops, the entry expires and traffic is routed elsewhere.

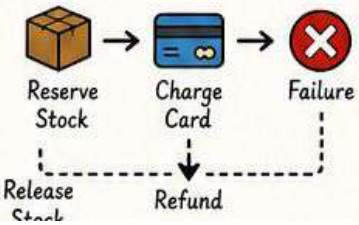
39



Q39. How do you handle clock skew?

A: Do not trust wall-clock ordering across machines. Use logical clocks, sequence numbers or a single sequencer per partition, and always store UTC timestamps.

40



Q40. What is a saga and when do you use it?

A: A long-running transaction split into local steps, each with a compensating action, used when a distributed two-phase commit is too costly. Example: reserve stock → charge card → on failure, refund and release stock.

SECTION 8 — RELIABILITY & RECAP

47 Q47. What do you monitor in a distributed system?

A: The four golden signals — latency, traffic, errors, saturation — plus business metrics and queue lag. Alert on user-visible SLOs, not on raw CPU.

48 Q48. What are SLI, SLO and SLA?

A: SLI is the measured value (p99 latency). SLO is the internal target (99.9% under 500 ms). SLA is the external contract with penalties. Error budget = 100% minus the SLO.

49 Q49. How do you deploy safely at scale?

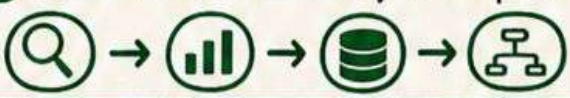
A: Blue-green or canary releases, feature flags, automatic rollback on SLO violation, backward-compatible database migrations (expand → migrate → contract).

50 Q50. How do you design for failure?

A: Assume every dependency fails. Add timeouts, bounded retries with jitter, circuit breakers, bulkheads, graceful degradation, and test it with chaos experiments.

ANSWER FRAMEWORK – 7 STEPS

1. Clarify scope and assumptions
2. Estimate scale (QPS, storage, bandwidth)
3. Define APIs and data model
4. Draw the high-level architecture
5. Deep dive the hardest component
6. Handle failures and bottlenecks
7. State the trade-offs you accepted



PHRASES THAT SCORE POINTS

- "This is read-heavy, so I will optimise reads first."
- "I will accept eventual consistency here, but not for payments."
- "That creates a hot partition — here is how I would split it."
- "I will make this idempotent so retries are safe."
- "If this service fails, the system degrades like this..."

COMMON MISTAKES

- ✗ Jumping to a diagram before clarifying requirements
- ✗ Never mentioning failure or trade-offs
- ✗ Naming technologies without justifying them
- ✗ Ignoring the interviewer's hints

